EECS 348 Assignment 4: GenAI Code Analysis

Student: Joshua Assefa

Course: EECS 348 – Software Engineering I (Fall 2026)

Date: October 15, 2026

1. GenAI Programs and Access Methods

- **GenAI Program 1:** Claude 3.5 Sonnet (Anthropic), accessed via the Claude web application interface (`(https://claude.ai)`) on my MacBook Air.
- **GenAI Program 2:** Microsoft Copilot (powered by OpenAI GPT-4), accessed via the Microsoft Copilot web interface (`(https://copilot.microsoft.com)`) on my MacBook Air.

2. Prompt Used for Both GenAIs

The exact same prompt below was given to both GenAI models:

> "Write a complete, single-file object-oriented C++ program (using classes and objects, not standalone procedural functions) that solves 9x9 Sudoku puzzles using recursive depth-first search with backtracking. For each empty cell, deduce a list of valid candidates (1-9) not present in the same row, column, or 3x3 block, and recursively try each candidate. The program must process five files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt, where digits 1-9 are pre-filled and '_' represents an empty cell. For each file, print the puzzle file name, the initial puzzle from the file, and all valid solutions found (since a puzzle may have multiple solutions). If no solution exists, print 'No solution found'."

3. Generated Code from Both GenAIs

Raw Output: GenAI 1 (Claude 3.5 Sonnet)

```
#include <iostream>

#include <fstream>

#include <vector>

#include <string>
```

```
class SudokuPuzzle {
private:
  int board[9][9];
  int solutionsFound;

  bool isSafe(int row, int col, int num) const {
    for (int x = 0; x < 9; x++) {
      if (board[row][x] == num || board[x][col] == num)
        return false;
    }
    int startRow = row - row % 3;
    int startCol = col - col % 3;
    for (int i = 0; i < 3; i++) {
      for (int j = 0; j < 3; j++) {
        if (board[i + startRow][j + startCol] == num)
          return false;
      }
    }
    return true;
  }

  std::vector<int> getCandidates(int row, int col) const {
    std::vector<int> candidates;
    for (int num = 1; num <= 9; num++) {
      if (isSafe(row, col, num)) {
```

```
            candidates.push_back(num);
        }
    }
    return candidates;
}

bool findEmpty(int &row, int &col) const {
    for (row = 0; row < 9; row++) {
        for (col = 0; col < 9; col++) {
            if (board[row][col] == 0)
                return true;
        }
    }
    return false;
}

void solve() {
    int row, col;
    if (!findEmpty(row, col)) {
        solutionsFound++;
        std::cout << "Solution " << solutionsFound << ":\n";
        print();
        std::cout << "\n";
        return;
    }
    std::vector<int> candidates = getCandidates(row, col);
```

```
        for (int num : candidates) {
            board[row][col] = num;
            solve();
            board[row][col] = 0;
        }
    }

public:
    SudokuPuzzle() : solutionsFound(0) {
        for (int i = 0; i < 9; i++)
            for (int j = 0; j < 9; j++)
                board[i][j] = 0;
    }

    bool load(const std::string &filename) {
        std::ifstream file(filename);
        if (!file.is_open()) return false;
        std::string val;
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                if (!(file >> val)) return false;
                board[i][j] = (val == "_") ? 0 : (val[0] - '0');
            }
        }
        return true;
    }
```

```
void print() const {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (board[i][j] == 0) std::cout << "_ ";
            else std::cout << board[i][j] << " ";
        }
        std::cout << "\n";
    }
}

void processFile(const std::string &filename) {
    std::cout << filename << "\n";
    if (!load(filename)) {
        std::cout << "Failed to load " << filename << "\n";
        return;
    }
    std::cout << "Initial Puzzle:\n";
    print();
    std::cout << "\n";
    solutionsFound = 0;
    solve();
    if (solutionsFound == 0) {
        std::cout << "No solution found\n\n";
    }
}
```

```
};


int main() {
    SudokuPuzzle solver;
    std::vector<std::string> files = {
        "puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"
    };
    for (const auto &f : files) {
        solver.processFile(f);
    }
    return 0;
}
```

Raw Output: GenAI 2 (Microsoft Copilot)

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class Sudoku {
public:
  char grid[9][9];

  bool loadPuzzle(std::string fileName) {
    std::ifstream fin(fileName);
    if (!fin) return false;
    for (int r = 0; r < 9; r++) {
      for (int c = 0; c < 9; c++) {
        fin >> grid[r][c];
      }
    }
    return true;
  }

  void display() {
    for (int r = 0; r < 9; r++) {
      for (int c = 0; c < 9; c++) {
        std::cout << grid[r][c] << " ";
      }
```

```
      std::cout << std::endl;
    }
}

bool valid(int r, int c, char ch) {
    for (int i = 0; i < 9; i++) {
        if (grid[r][i] == ch) return false;
        if (grid[i][c] == ch) return false;
        if (grid[3 * (r / 3) + i / 3][3 * (c / 3) + i % 3] == ch) return false;
    }
    return true;
}

bool solve() {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (grid[r][c] == '_') {
                for (char ch = '1'; ch <= '9'; ch++) {
                    if (valid(r, c, ch)) {
                        grid[r][c] = ch;
                        if (solve()) return true;
                        grid[r][c] = '_';
                    }
                }
                return false;
            }
```

```
            }
        }
        return true;
    }
};

int main() {
    std::string files[5] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
    for (int i = 0; i < 5; i++) {
        Sudoku s;
        if (s.loadPuzzle(files[i])) {
            std::cout << files[i] << std::endl;
            s.display();
            if (s.solve()) {
                std::cout << "Solved:" << std::endl;
                s.display();
            } else {
                std::cout << "No solution found" << std::endl;
            }
        }
    }
    return 0;
}
```

4. Code Analysis Across Lecture Metrics

a. Correctness

- **Claude 3.5 Sonnet:** High correctness. It properly parses whitespace-delimited tokens (1–9 and _), deduces a std::vector<int> list of valid candidates per empty cell as specified in the assignment prompt, and continues backtracking after reaching a base-case solution (!findEmpty(row, col)) instead of returning true immediately. Because it does not short-circuit on the first solution, it accurately discovers and prints **both** solutions for puzzle3.txt and outputs "No solution found" for puzzle2.txt and puzzle4.txt. One minor edge-case omission: isSafe() assumes pre-filled cells do not conflict with each other prior to search, though none of the 5 test files contain duplicate pre-filled numbers in the same unit.
- **Microsoft Copilot:** Moderate-to-low correctness relative to the specification. While it solves single-solution puzzles (puzzle1.txt and puzzle5.txt) and detects unsolvable puzzles (puzzle2.txt and puzzle4.txt), its solve() method returns true immediately upon finding the first valid configuration (if (solve()) return true;). Consequently, it fails the requirement: *"if there is more than one solution to the puzzle, your program should print them all out"*, printing only 1 of the 2 valid solutions for puzzle3.txt. Additionally, it does not construct an explicit candidate list before recursing.

b. Execution Time

- **Claude 3.5 Sonnet:** In the worst case, depth-first search with backtracking on k empty cells has an upper-bound time complexity of $O(9^k)$, though constraint pruning (isSafe) drastically reduces the branching factor at each level. For each empty cell, findEmpty scans at most 81 cells ($O(1)$ bounded grid size), and getCandidates checks 9 digits across 27 row/column/block cells ($9 \times 27 = 243$ operations, or $O(1)$ constant overhead per node). Across all five puzzle files, execution completes in under 0.02 seconds on the Cycle server.
- **Microsoft Copilot:** Also runs in $O(9^k)$ worst-case exponential time with pruned branches. Because it avoids allocating a std::vector<int> at each recursion step and tests characters '1' to '9' inline, it avoids dynamic heap allocation overhead per node. However, because it restarts the nested for (int r = 0; r < 9; r++) loop from $(0,0)$ at every recursive call, it performs redundant scans over already-filled upper rows.

c. Space Complexity

- **Claude 3.5 Sonnet:** The recursion call stack depth is bounded by the number of empty cells $k \leq 81$, giving $O(k)$ stack frames (which is $O(1)$ auxiliary space for a fixed 9×9 grid). At each active stack frame, std::vector<int> candidates holds at most 9 integers (≤ 36 bytes). Thus, total auxiliary memory is strictly bounded at $\leq 81 \times 36$ bytes ≈ 2.9 KB.

- **Microsoft Copilot:** Stores a fixed 9×9 char array (81 bytes) and uses $O(k)$ call stack frames with only local loop counters (r, c, ch), resulting in slightly smaller constant stack frame size, though both are $O(1)$ with respect to the fixed board dimensions.

d. Maintainability

- **Claude 3.5 Sonnet:** Good encapsulation (private data members and helper methods, public interface methods), clear method naming (getCandidates, findEmpty, isSafe), and const correctness on read-only methods. However, both board representation and solver backtracking state are coupled inside a single SudokuPuzzle class.
- **Microsoft Copilot:** Poor encapsulation. All member variables (char grid[9][9]) and helper routines are declared public, violating object-oriented information hiding. Method and variable names (s, ch, fin, valid) are overly terse, and no comments or documentation are provided.

5. Selection and Justification

- **Selected Baseline Program: Claude 3.5 Sonnet (GenAI 1)**.
- **Justification:** Claude 3.5 Sonnet is the only model that satisfied two core functional requirements out of the box: (1) it explicitly deduces a candidate list (getCandidates) for each empty cell before recursing as described in the specification, and (2) it exhaustively searches the entire state space to print **all** valid solutions rather than terminating after the first solution (essential for puzzle3.txt, which has 2 solutions). It also uses proper private/public access modifiers and const-qualified methods.

6. Code Improvements Implemented

To elevate the Claude 3.5 Sonnet baseline to production quality across all four metrics, I implemented the following improvements in Assignment4.cpp:

1. **Correctness (Initial Board Validation & Self-Exclusion in Constraint Checks):** Added isInitialBoardValid() and modified isValidPlacement(int row, int col, int val) to exclude the cell (row, col) itself when checking row, column, and 3×3 block peers. This prevents false self-collisions and guarantees that if an input file ever contains a pre-existing duplicate digit in a row, column, or block, the solver immediately rejects it and outputs "No solution found" rather than wasting time or crashing. I also added the "Solutions found: X" summary line to match solution1.txt.
2. **Execution Time (Short-Circuit Peer Scanning):** Separated the row, column, and 3×3 block checks in isValidPlacement() so that a duplicate in the row or column immediately returns false without computing block offsets or iterating over the 3×3 sub-grid.

3. **Space Complexity (Separation of Board State from Solver State):** Separated the board storage (SudokuBoard) from the recursive solver (SudokuSolver), passing SudokuBoard& by reference across recursive calls so the 324-byte grid is never copied on the call stack.
4. **Maintainability (Object-Oriented Decoupling & Full Documentation):** Refactored the monolithic class into two single-responsibility classes: SudokuBoard (managing grid representation, file I/O, printing, and candidate deduction) and SudokuSolver (managing the recursive DFS backtracking algorithm and solution counting). Finally, I added the complete prologue header, block-level source attributions with URLs, and line-by-line comments on every line of code.